



PES
UNIVERSITY

CELEBRATING 50 YEARS

Data Structures

Dilip Kumar Maripuri
Computer Applications



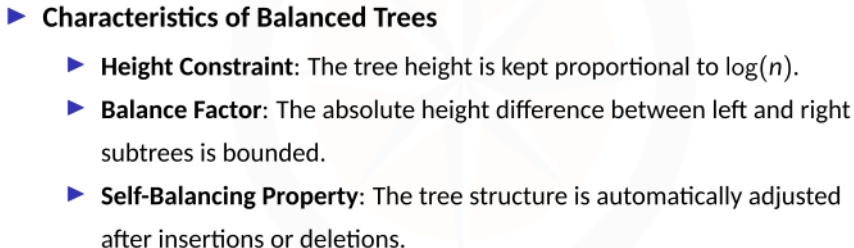


Data Structures

Session : AVL Trees

Dilip Kumar Maripuri
Computer Applications







Data Structures

Types of Balanced Trees

► AVL Trees

- Named after **Adelson-Velsky and Landis** (1962).
- The **balance factor** is defined as:

Balance Factor = Height of Left Subtree — Height of Right Subtree

- Balance factor must be $-1, 0, +1$ for every node.
- If violated, **rotations (LL, RR, LR, RL)** restore balance.



▶ Red-Black Trees

- ▶ A BST where each node is assigned a **red** or **black** color.
- ▶ Ensures balance using **coloring rules**:
 1. Root is always **black**.
 2. No two consecutive **red nodes**.
 3. Every path from root to a leaf has the **same number of black nodes**.
- ▶ Used in **C++ STL (map, set, multimap, multiset)** and **Linux kernel**.



- ▶ These are types of balanced search trees that allow more than one element in a node, falling under the representation-change variety of the strategy.
- ▶ In a **2-3 tree**, every internal node may contain either one or two values and have either two or three children, respectively.
- ▶ In a **2-3-4 tree**, every internal node can have between two and four children and contain one to three elements.
- ▶ **B-trees** generalize these concepts, allowing for more than two children per node and perfect balance across the tree.



Data Structures

Tree Rotations

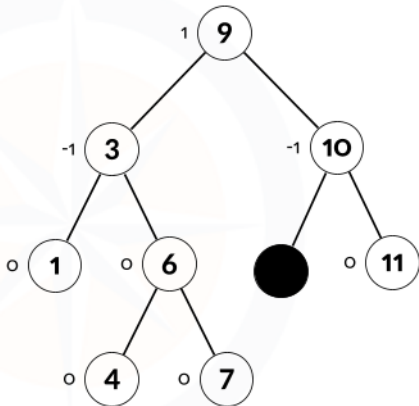
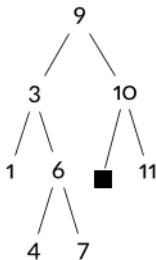
- ▶ A critical operation in maintaining balance in certain types of self-balancing binary trees (such as AVL trees and red-black trees) is the tree rotation.
- ▶ A rotation is a local operation in a search tree that preserves the in-order ordering of the elements.
- ▶ If an insertion or deletion violates the tree's balance properties, rotations are used to restore balance.
 - ▶ **Left Rotation (LL)**
 - ▶ **Right Rotation (RR)**
 - ▶ **Left-Right Rotation (LR)**
 - ▶ **Right-Left Rotation (RL)**



- ▶ An AVL tree is a binary search tree in which the balance factor of every node, which is defined as the difference between the heights of the node's left and right subtrees, is either 0 or +1 or -1.
- ▶ The height of the empty tree is defined as -1.
- ▶ The balance factor can also be computed as the difference between the numbers of levels rather than the height difference of the node's left and right subtrees.

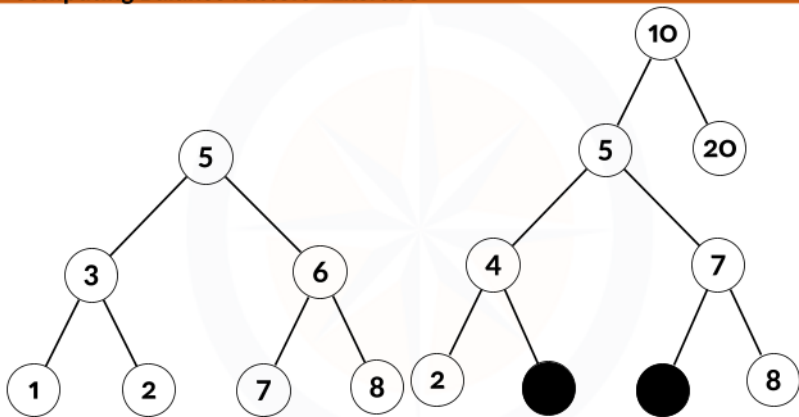


Computing Balance Factors





Computing Balance Factors - Exercise



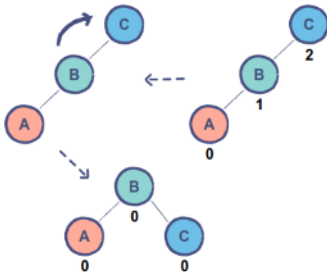
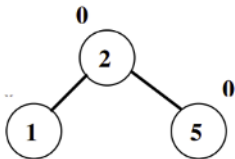
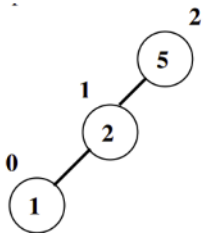


- ▶ If an insertion of a new node makes an AVL tree unbalanced, we transform the tree by a rotation.
- ▶ A rotation in an AVL tree is a local transformation of its sub-tree rooted at a node whose balance has become either +2 or -2.
- ▶ If there are several such nodes, we rotate the tree rooted at the unbalanced node that is the closest to the newly inserted leaf



Data Structures

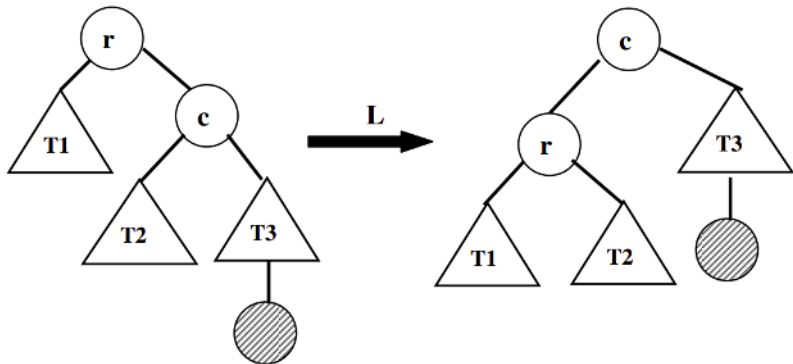
Single Right Rotation





Data Structures

Single Left Rotation

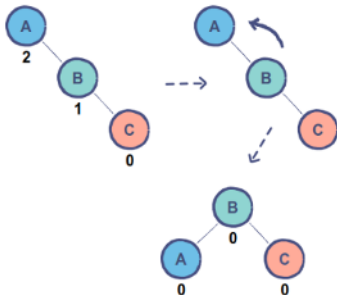
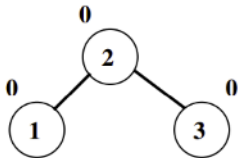
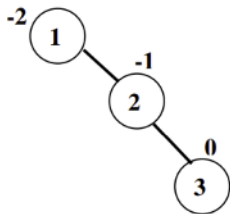


- Here rotation is performed after a new key is inserted into the right sub-tree of the right child of a tree whose root had the balance of -1 before the insertion



Data Structures

Single Left Rotation





Data Structures

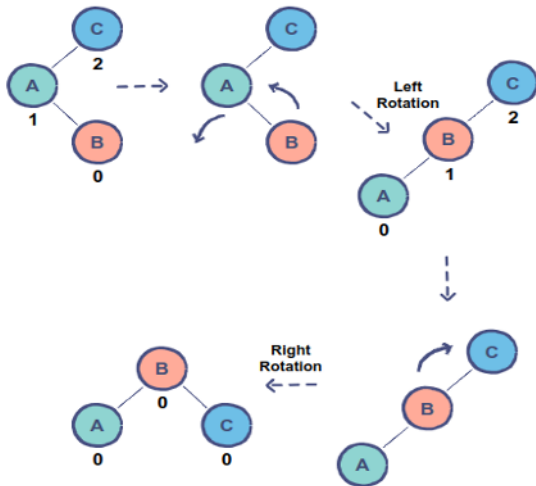
Double – Left Right Rotation

- ▶ **Combination of two rotations**
 - ▶ perform left rotation of the left sub-tree of root r
 - ▶ perform right rotation of the new tree rooted at r
- ▶ It is performed after a new key is inserted into the **right sub-tree of the left child of a tree whose root had the balance of +1** before the insertion



Data Structures

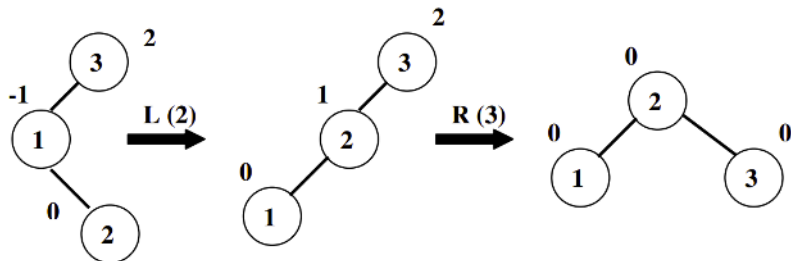
Double - Left Right Rotation





Data Structures

Double – Left Right Rotation

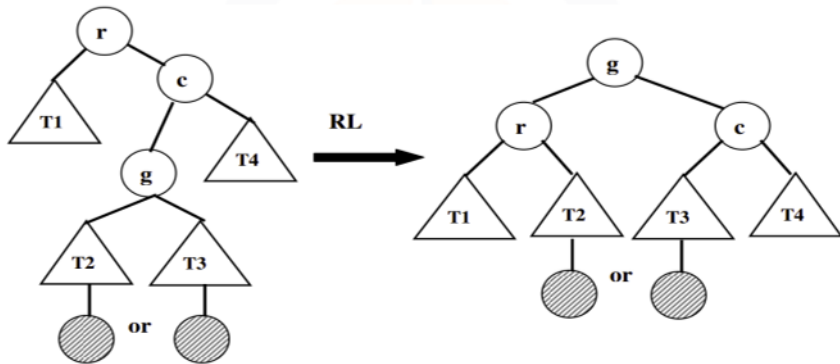






Data Structures

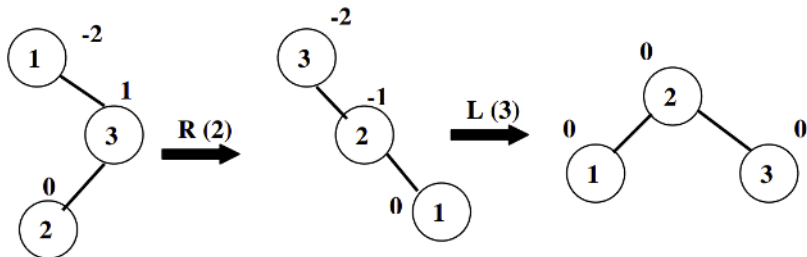
Double - Right Left Rotation





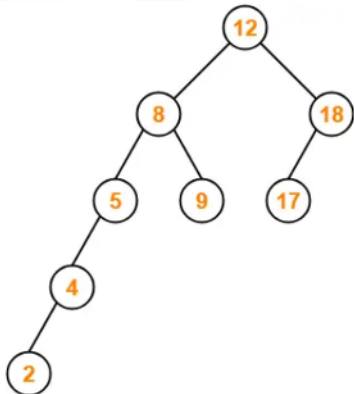
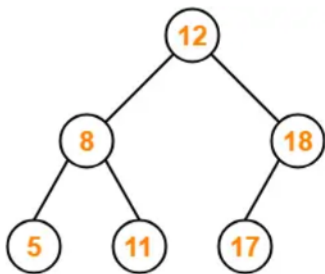
Data Structures

Double – Right Left Rotation





Data Structures Illustrations





Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026